Compte rendu, TP 3 : Enregistrer modèles et artefacts dans MLflow

Nom : Houqqa Fatima Zahra | **Date :** 14/10/2026 | **Cours :** Déploiement d'applications intelligentes, MLOps

Étape 1 : exécutions journalisées dans MLflow

Trois runs ont été lancés dans l'expérience `tp3-registre` avec `src/train_complet.py`. Le script journalise les paramètres, les métriques, les tags (auteur, commit Git, jeu de données), deux artefacts et le modèle avec sa signature.

Run	n_estimators	max_depth	F1
rf-200-8	200	8	0.8549
rf-300-12	300	12	0.8636
rf-500-16	500	16	0.8669

Script `src/train_complet.py` :

```
import sys, json, subprocess, mlflow, mlflow.sklearn, pandas as pd
import matplotlib; matplotlib.use('Agg')
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import (f1_score, precision_score, recall_score,
                             roc_auc_score, ConfusionMatrixDisplay)

n_estimators = int(sys.argv[1]); max_depth = int(sys.argv[2])
mlflow.set_tracking_uri('http://127.0.0.1:5000')
mlflow.set_experiment('tp3-registre')

df = pd.read_csv('data/transactions.csv')
X, y = df.drop(columns='fraude'), df['fraude']
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=42)

with mlflow.start_run(run_name=f'rf-{n_estimators}-{max_depth}'):
    m = RandomForestClassifier(n_estimators=n_estimators,
max_depth=max_depth,
                               random_state=42).fit(Xtr, ytr)
    pred = m.predict(Xte); proba = m.predict_proba(Xte)[:, 1]
```

```
    mlflow.log_params({'n_estimators': n_estimators, 'max_depth': max_depth})
    mlflow.log_metrics({'f1': f1_score(yte, pred),
                        'precision': precision_score(yte, pred),
                        'recall': recall_score(yte, pred),
                        'roc_auc': roc_auc_score(yte, proba)})
    commit = subprocess.getoutput('git rev-parse --short HEAD')  #
tracabilite
    mlflow.set_tags({'auteur': 'votre-nom', 'commit': commit,
                     'jeu_de_donnees': 'transactions v1'})

    ConfusionMatrixDisplay.from_predictions(yte, pred)            # artefact
1
    plt.savefig('reports/confusion.png', dpi=110, bbox_inches='tight')
    mlflow.log_artifact('reports/confusion.png')

    imp = dict(zip(X.columns, m.feature_importances_.round(4)))   # artefact
2
    json.dump(imp, open('reports/importances.json', 'w'), indent=2)
    mlflow.log_artifact('reports/importances.json')

    mlflow.sklearn.log_model(m, name='model', input_example=Xte[:3])  #
signature
    print('F1 =', round(f1_score(yte, pred), 4))
```

Artefacts du run rf-500-16 (onglet Artifacts)

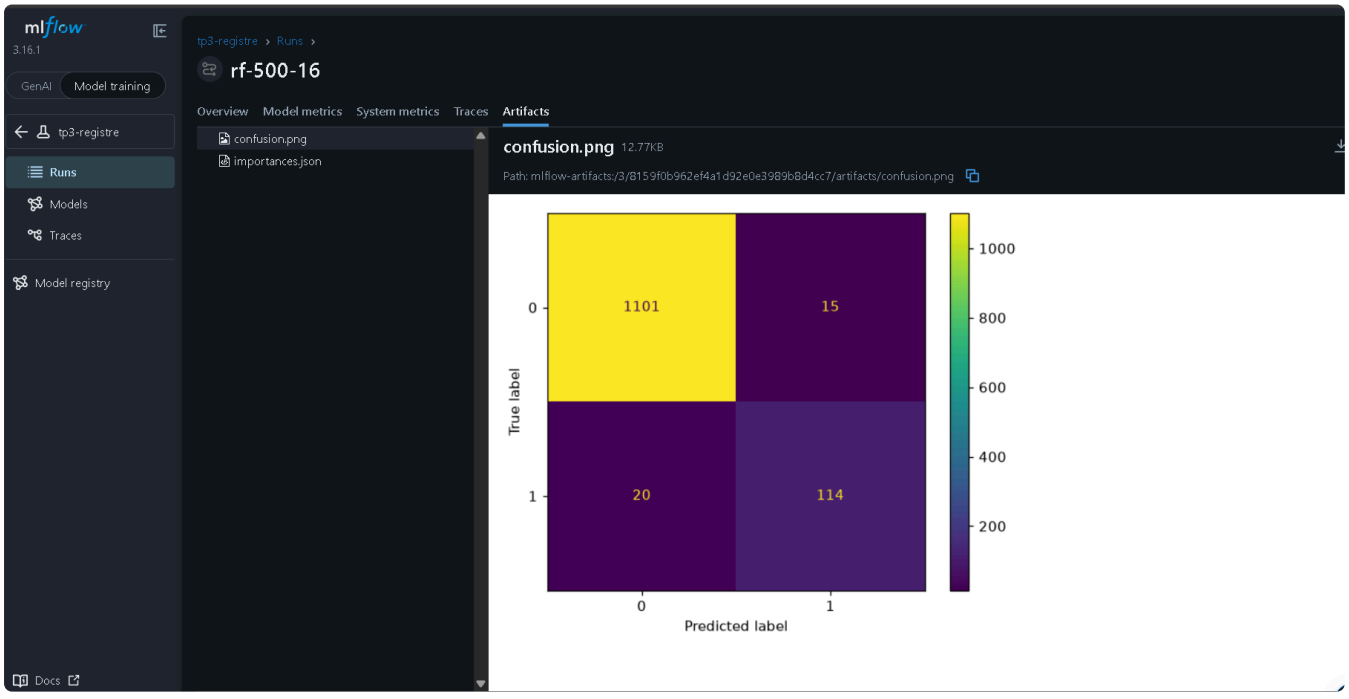


Figure 1 : `confusion.png` (1101 vrais négatifs, 15 faux positifs, 20 faux négatifs, 114 vrais positifs).

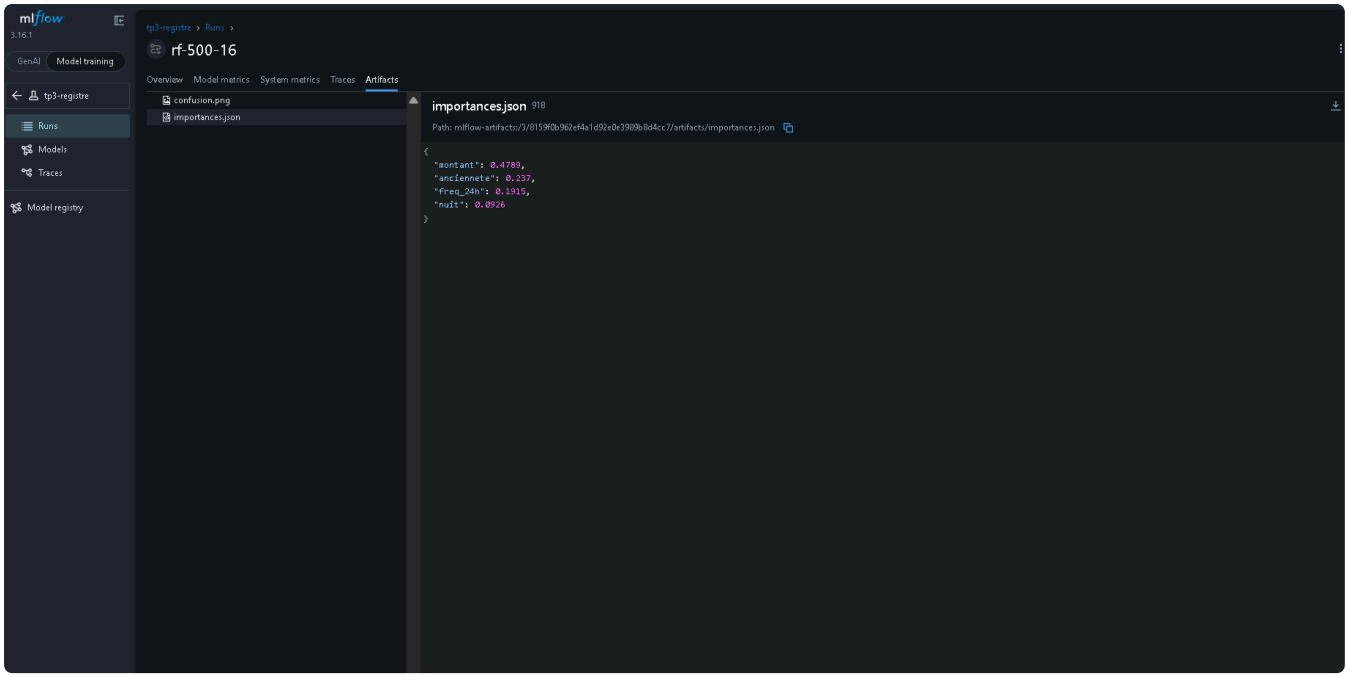


Figure 2 : `importances.json` (montant 0.4789, anciennete 0.237, freq_24h 0.1915, nuit 0.0926).

Étapes 3 et 4 : registre et alias

Les trois runs ont été enregistrés comme versions 1 à 3 du modèle `fraude`, classées par F1 décroissant. Deux alias ont ensuite été posés : `staging` sur la version 2 et `production` sur la version 1, qui porte la description « validee par le metier le 12/03 ».

Version	Run source	F1	Alias
1	rf-500-16	0.8669	`production`
2	rf-300-12	0.8636	`staging`
3	rf-200-8	0.8549	aucun

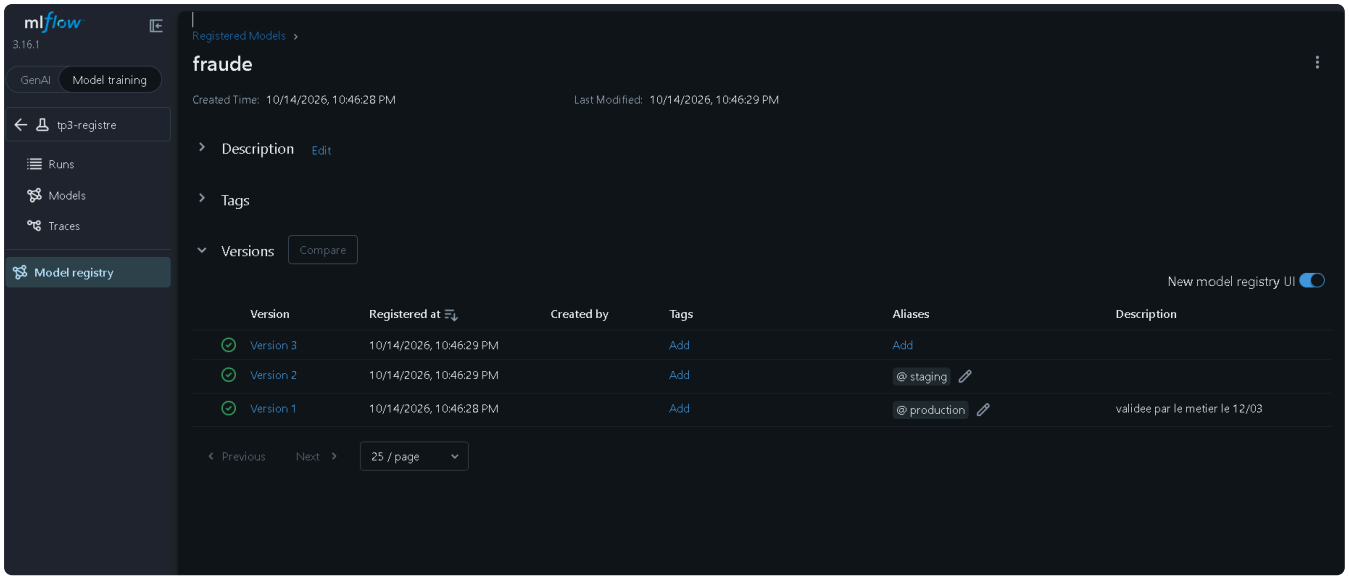


Figure 3 : registre du modèle `fraude` avec ses trois versions et les alias `staging` (version 2) et `production` (version 1).

Étape 5 : retour arrière par déplacement d'alias

L'alias `production` a été déplacé sur la version 2, puis ramené sur la version 1. Le code client est identique entre les deux appels de `predict` : seule l'affectation de l'alias change, sans reconstruction ni redéploiement. Les deux appels renvoient la même prédiction.

```
servi : [1]
servi : [1]
```

La signature a aussi joué son rôle ici : une première tentative avec `montant = 8500` (entier) a été refusée (`Can not safely convert int64 to float64`), car la signature impose un `double` . Avec `montant = 8500.0` , l'appel passe.

Étapes 6 et 7 : service HTTP et requête invalide

Le modèle promu a été servi en HTTP avec `mlflow models serve -m 'models:/fraude@production' -p 8000 --env-manager local` .

Appel valide (4 colonnes) :

```
Invoke-RestMethod -Method Post -Uri http://127.0.0.1:8000/invocations `
  -ContentType 'application/json' -Body $corps

predictions
-----------
{1}
```

Appel invalide (colonne `montant` seule) :

```
{"error_code": "INVALID_PARAMETER_VALUE", "message": "Failed to predict data
'   montant\n0   8500.0'. \nError: Failed to enforce schema of data '
montant\n0   8500.0' with schema '['montant': double (required),
'anciennete': long (required), 'freq_24h': long (required), 'nuit': long
(required)]'. Error: Model is missing inputs ['anciennete', 'freq_24h',
'nuit'].", "sqlstate": "KAM01", "error_class": "SCHEMA_ENFORCEMENT_FAILED"}
```

La signature bloque la requête avant le modèle et nomme les colonnes manquantes : `anciennete`, `freq_24h` et `nuit`.

Question 1 : Trois éléments du dossier `model` (en plus du fichier sérialisé)

- **`MLmodel`** : décrit les *flavors* (python_function, sklearn) et la **signature** du modèle (nom et type de chaque colonne d'entrée). Il permet de charger et d'appeler le modèle sans deviner le format des données.
- **`conda.yaml`** / **`python_env.yaml`** : décrivent l'environnement d'exécution (version de Python et dépendances). Ils permettent de recréer le même environnement sur une autre machine.
- **`requirements.txt`** : liste les paquets et leurs versions (scikit-learn, pandas, etc.). Il permet de réinstaller exactement les mêmes dépendances et d'éviter les incompatibilités.

Question 2 : Différence entre une version et un alias

Une **version** est un numéro fixe et immuable : la version 1 reste toujours le même modèle. Un **alias** (`staging`, `production`) est une étiquette mobile qui pointe vers une version. Le service d'inférence utilise l'alias (`models:/fraude@production`) afin que le changement de modèle, ou un retour arrière, se fasse **en déplaçant l'alias**, sans reconstruire, sans redéployer et sans modifier le code du client. C'est ce que démontre l'étape 5 : le code client est identique avant et après le déplacement de l'alias.

Question 3 : Informations mobilisées pour expliquer une décision prise il y a six mois

On mobilise :

- les **paramètres** du run (`n_estimators` , `max_depth`) ;
- les **métriques** (F1, précision, rappel, ROC-AUC) ;
- les **artefacts** (matrice de confusion, importances des variables) ;
- les **tags** (auteur, commit Git, version du jeu de données) ;
- la **description** de la version dans le registre (par exemple « validée par le métier le 12/03 ») ;
- l'**historique des alias**, qui indique quelle version était en production et à quel moment.

Question 4 : Pourquoi journaliser le commit Git et la version du jeu de données ?

Pour relier le modèle au **code** et aux **données** exacts qui l'ont produit. Sans ces informations, on ne peut ni **reproduire** l'entraînement, ni comprendre pourquoi les performances ont changé (nouveau code ou nouvelles données ?). C'est la base de la **traçabilité** et de l'audit.

Question 5 : Intérêt de la signature pour l'équipe qui développera l'API (TP 10)

La signature est un **contrat d'entrée** : elle précise les colonnes attendues, leurs types (`montant` en double, les autres en entier) et leur caractère obligatoire. L'équipe API peut ainsi construire ses requêtes et sa validation sans deviner le format. En cas d'entrée invalide, l'erreur est **explicite**, par exemple `Model is missing inputs ['anciennete', 'freq_24h', 'nuit']` (observée à l'étape 7), au lieu d'un message obscur survenant plus loin dans le modèle.

Remarque

Avec MLflow 3.x, le modèle packagé n'apparaît plus sous l'artefact `model` du run, mais dans l'onglet *Models* (entité « Logged Model »), avec le même contenu (MLmodel, signature, conda.yaml, model.pkl, etc.).

Livrables joints

- `src/train_complet.py`
- Capture de l'onglet **Artifacts** (`confusion.png` + `importances.json`)
- Capture de l'onglet **Models** (3 versions + 2 alias)
- Trace de l'**appel valide** (`predictions : {1}`) et de l'**appel invalide** (`SCHEMA_ENFORCEMENT_FAILED`)